

EPIC VI

PROGRAMMING SCHOOL FOR SCIENTIFIC RESEARCH

3 - 7 AUGUST 2026

Physically Informed Neural Networks for solving PDEs

Sesión 2

Poisson 2D

PyTorch

Pablo Rodríguez López



Universidad
Rey Juan Carlos



Redes neuronales informadas por la física

De aproximar datos a satisfacer ecuaciones diferenciales

Sesión 2

PINNs

PyTorch

Objetivos de la sesión

1 • Representar

Interpretar una red como función $u_\theta(x,y)$.

2 • Derivar

Usar autodiferenciación para gradientes y Laplaciano.

3 • Formular

Construir pérdidas de física y contorno.

4 • Entrenar

Optimizar parámetros con Adam en CPU.

5 • Diagnosticar

Separar entrenamiento y validación.

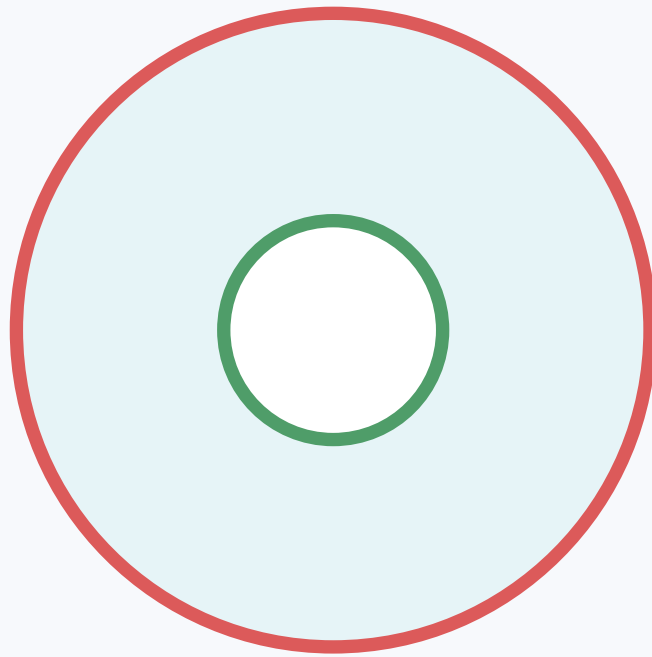
6 • Comparar

Contrastar PINN y FEM con prudencia.

Pregunta guía

¿Puede una red neuronal aprender una solución compatible con una EDP?

Problema modelo: Poisson en un anillo



Γ_D - exterior

Γ_N - interior

$$\begin{aligned}\Delta u &= f & \forall \mathbf{r} \in \Omega \\ u &= g & \forall \mathbf{r} \in \Gamma_D \\ \partial_n u &= h & \forall \mathbf{r} \in \Gamma_N\end{aligned}$$

$$f(x, y) = x$$

$$g(x, y) = \sin(\pi y)$$

$$h(x, y) = \sqrt{x^2 + 1}$$

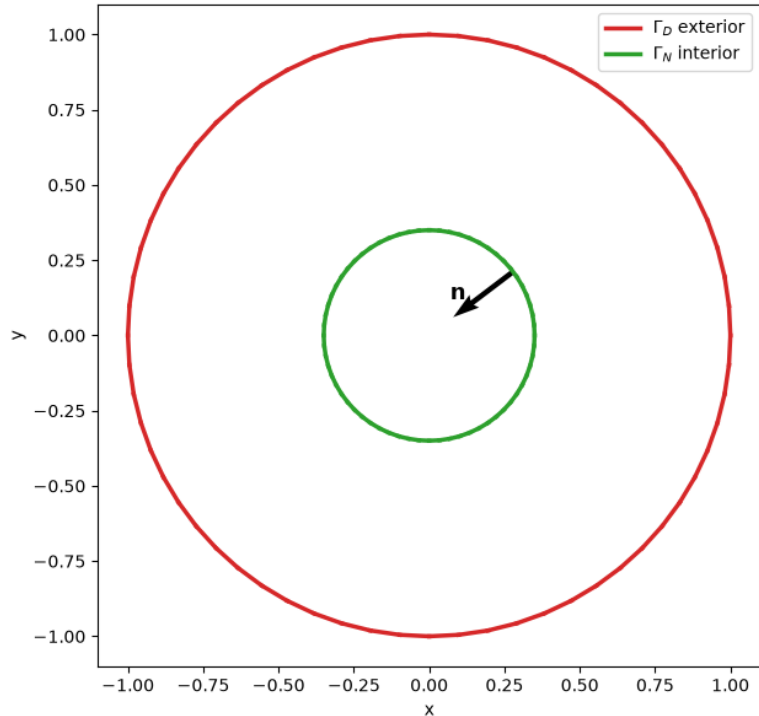
Mismo problema, nueva representación

$$u_h(\mathbf{r}) = \sum_{n=1}^N U_n \varphi_n(\mathbf{r}) \longrightarrow u_\theta(\mathbf{r}) = \text{NN}_\theta(\mathbf{r})$$

Cambia la familia aproximante; la EDP y los datos no cambian.

Problema modelo: Poisson en un anillo

Dominio Ω , fronteras y normal interior



$$\begin{aligned}\Delta u &= f & \forall \mathbf{r} \in \Omega \\ u &= g & \forall \mathbf{r} \in \Gamma_D \\ \partial_n u &= h & \forall \mathbf{r} \in \Gamma_N\end{aligned}$$

$$f(x, y) = x$$

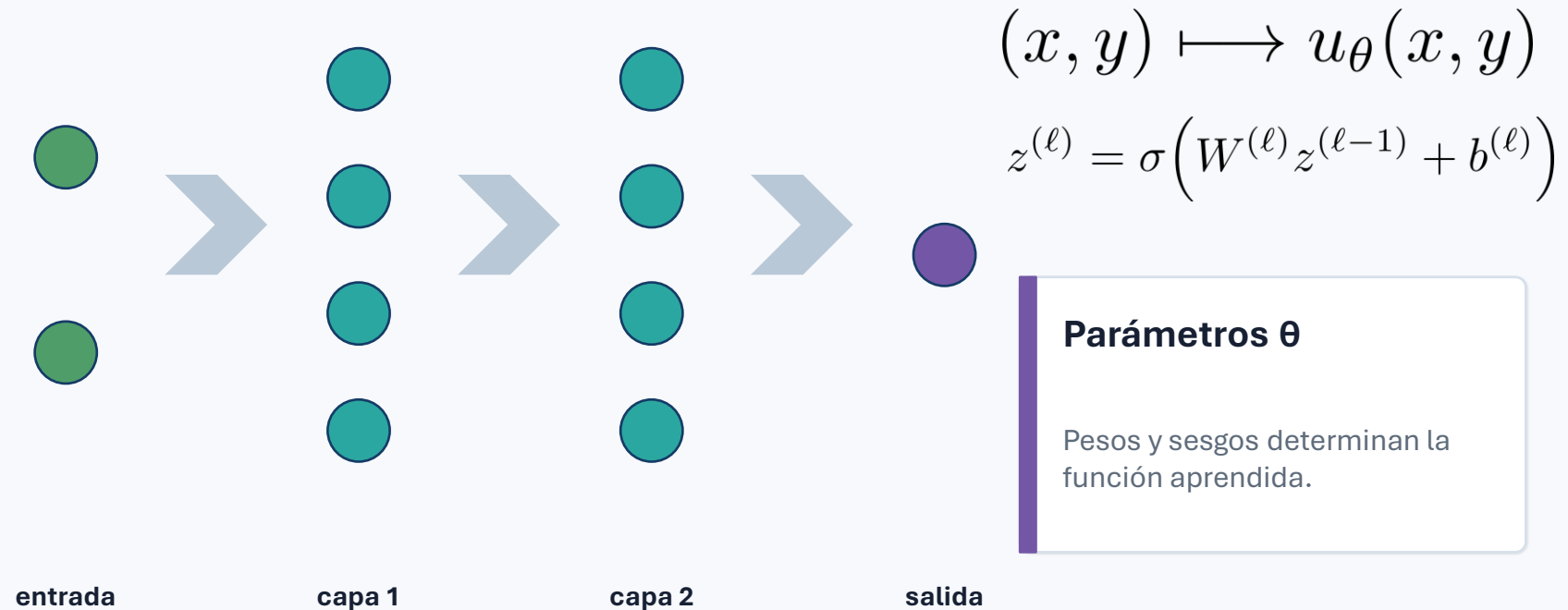
$$g(x, y) = \sin(\pi y)$$

$$h(x, y) = \sqrt{x^2 + 1}$$

En el agujero, la normal exterior apunta hacia el centro.

$$u_h(\mathbf{r}) = \sum_{n=1}^N U_n \varphi_n(\mathbf{r}) \longrightarrow u_\theta(\mathbf{r}) = \text{NN}_\theta(\mathbf{r})$$

Una red neuronal es una función parametrizada



Anchura, profundidad y activación controlan la familia de funciones.

La activación debe admitir segundas derivadas

Tanh

suave y acotada

Softplus

suave y positiva

Sigmoid

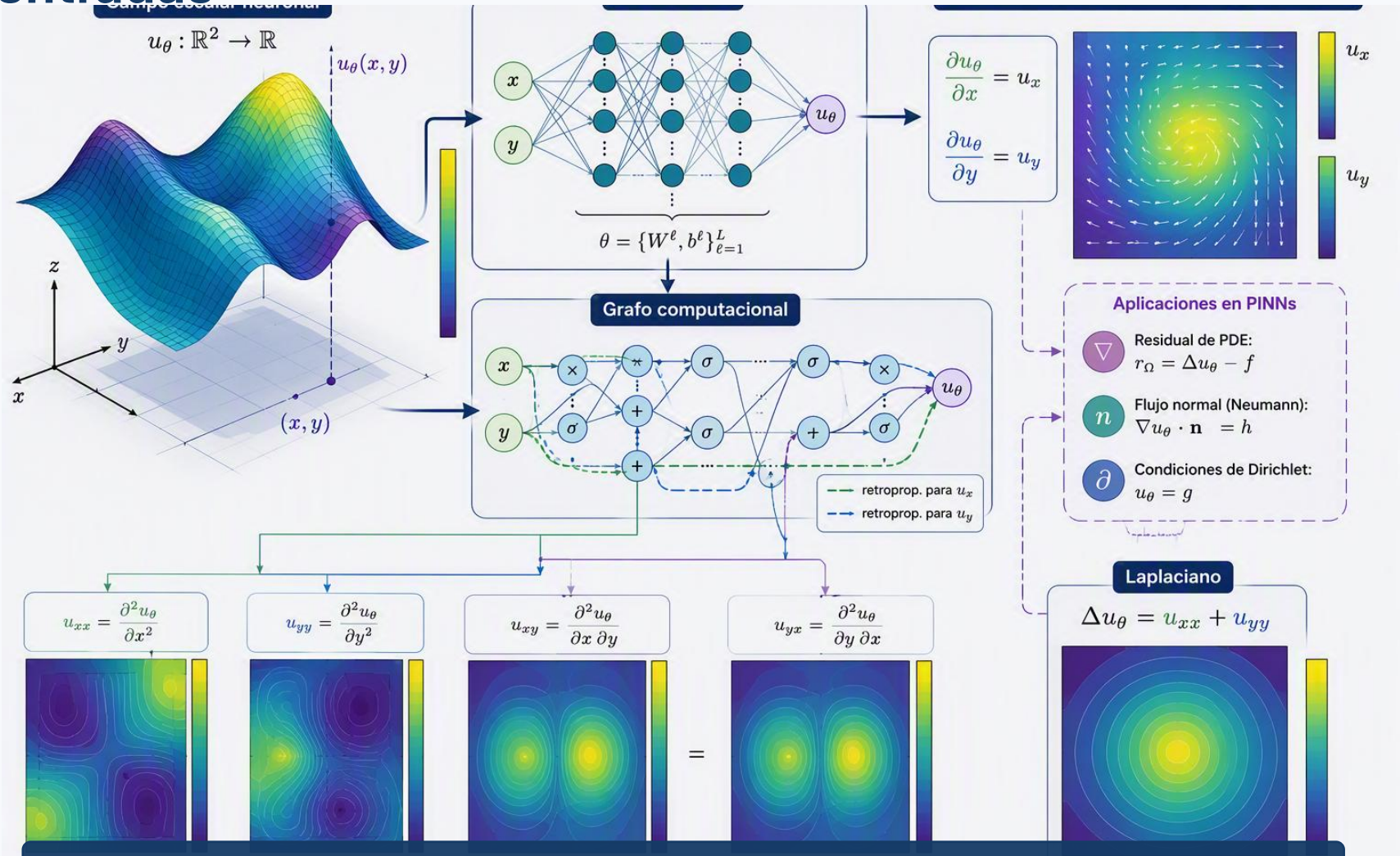
suave, puede saturar

ReLU

segunda derivada nula casi en todas partes

$$\Delta u_\theta = \frac{\partial^2 u_\theta}{\partial x^2} + \frac{\partial^2 u_\theta}{\partial y^2}$$

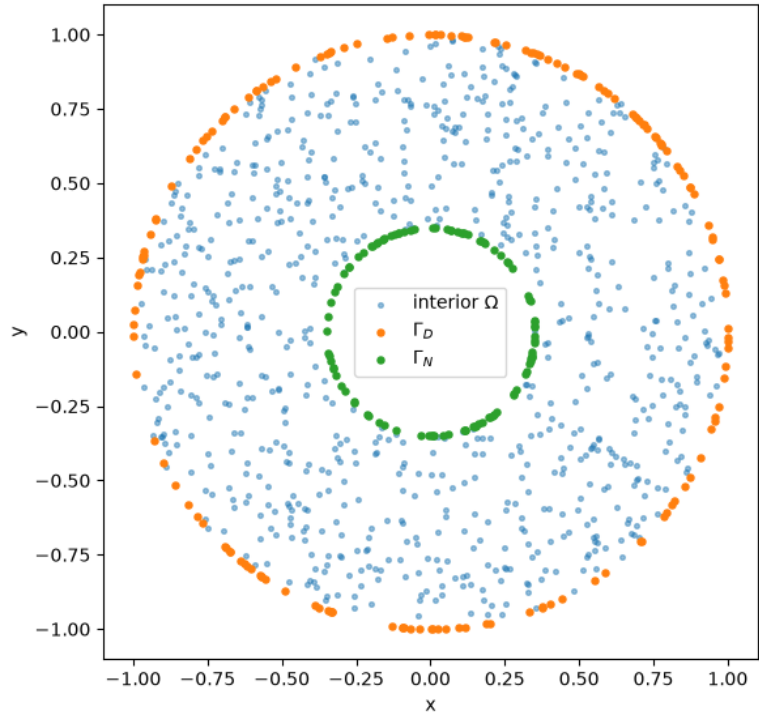
Autodiferenciación: derivar la red respecto a las entradas



PyTorch construye el grafo y aplica la regla de la cadena hasta obtener ∇u_θ y Δu_θ .

Muestrear el dominio forma parte del método

Puntos de colocación



Interior Ω

Muestreo uniforme en área, no uniforme en radio.

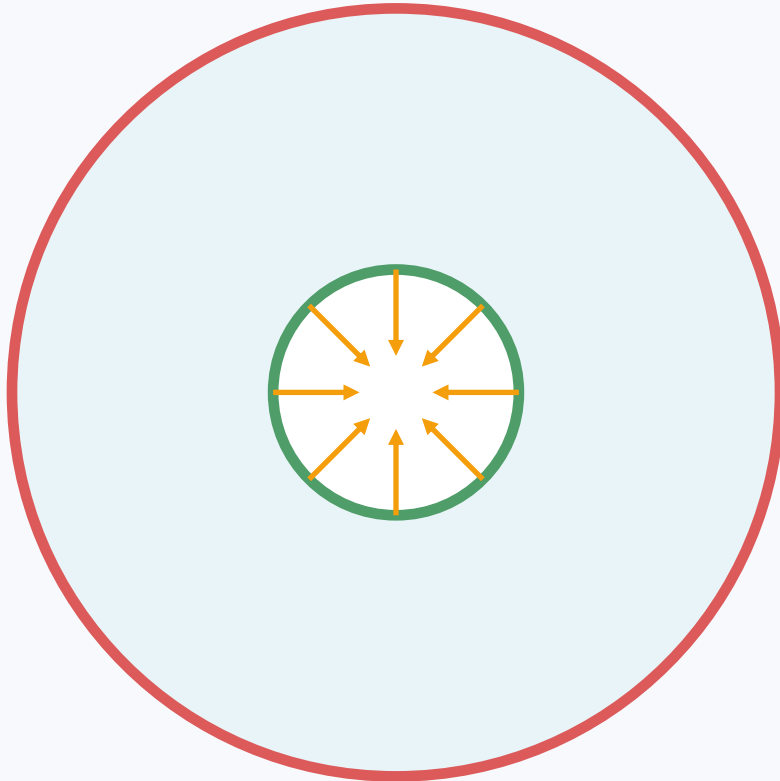
Frontera Γ_D

Puntos en la circunferencia exterior.

Frontera Γ_N

Puntos en la circunferencia interior.

Orientación de la normal en el agujero



$$\mathbf{n} = -\frac{(x, y)}{\sqrt{x^2 + y^2}} \quad \forall (x, y) \in \Gamma_N$$

$$\partial_n u_\theta = \nabla u_\theta \cdot \mathbf{n} = h$$

Cambiar la orientación cambia el signo del flujo normal y altera la pérdida.

Residual interior

$$r_{\Omega}(\mathbf{r}) = \Delta u_{\theta}(\mathbf{r}) - f(\mathbf{r})$$

$$\mathcal{L}_{\Omega} = \frac{1}{N_{\Omega}} \sum_{n=1}^{N_{\Omega}} |r_{\Omega}(\mathbf{r}_n)|^2$$

Minimizar $\mathcal{L}_{\Omega}$ no garantiza por sí solo las condiciones de contorno.

Condiciones de contorno como pérdidas

Dirichlet

$$\mathcal{L}_D = \frac{1}{N_D} \sum_{j=1}^{N_D} |u_\theta(\mathbf{r}_j) - g(\mathbf{r}_j)|^2$$

La red se penaliza cuando u_θ difiere de g sobre Γ_D .

Neumann

$$\mathcal{L}_N = \frac{1}{N_N} \sum_{k=1}^{N_N} |\partial_n u_\theta(\mathbf{r}_k) - h(\mathbf{r}_k)|^2$$

La red se penaliza cuando el flujo normal difiere de h sobre Γ_N .

Blanda significa aproximada, no exacta.

Pérdida total y balance de objetivos

$$\mathcal{L} = \lambda_{\Omega} \mathcal{L}_{\Omega} + \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N$$



física interior



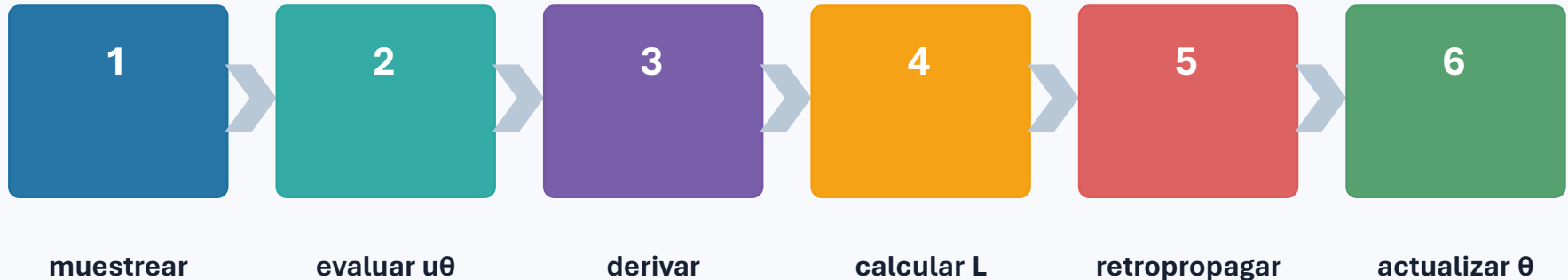
Dirichlet



Neumann

Los pesos cambian la solución del problema de optimización, no solo la escala de la gráfica.

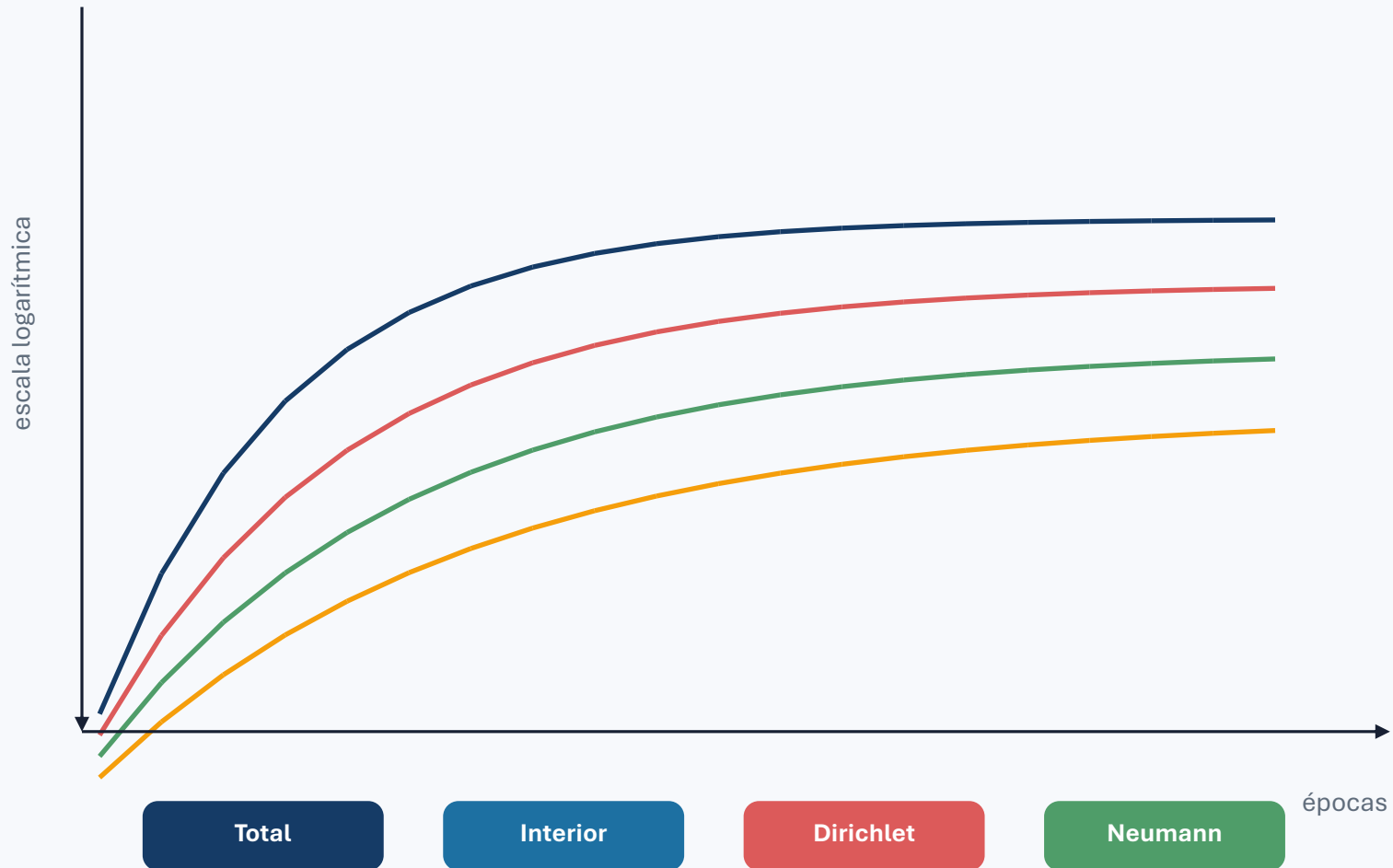
Entrenamiento con Adam



$$\theta \leftarrow \theta - \alpha \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

La pérdida puede descender sin que la solución sea científicamente fiable.

Qué observamos durante el entrenamiento



Validar en puntos nuevos

Residual interior RMS

¿satisface la EDP?

Dirichlet RMS / máximo

¿respeta g ?

Neumann RMS / máximo

¿respeta h ?

Mapa espacial

¿dónde falla?

Usar los mismos puntos de entrenamiento produce una visión demasiado optimista.

PINN y FEM: misma EDP, algoritmos distintos

FEM	PINN
Representación	base local red global
Problema numérico	sistema lineal optimización no convexa
Dirichlet	algebraicamente penalización blanda
Derivadas	forma débil autodiferenciación
Reproducibilidad	alta depende de semilla

Comparar requiere usar el mismo problema, puntos comunes y métricas independientes.

Deep Ritz: aprender minimizando una energía

$$J[u] = \int_{\Omega} d\mathbf{r} \left(\frac{|\nabla u|^2}{2} + uf \right) - \oint_{\Gamma_N} ds hu$$

Ventaja

Necesita primeras derivadas,
no segundas.

Coste

Las integrales se aproximan
mediante muestreo.

Dirichlet

Sigue requiriendo
penalización o construcción
exacta.

PINN fuerte y Deep Ritz son formulaciones diferentes del mismo problema físico.

Problemas inversos: aprender también parámetros físicos

$$\kappa \Delta u = f \Rightarrow u_\theta, \kappa \leftarrow \min (\mathcal{L}_{\text{Física}} + \mathcal{L}_{\text{datos}})$$

Observaciones

valores interiores de u

Física

residual de la EDP

Parámetro

α desconocido y positivo

Ruido

afecta identificabilidad

Una solución visualmente plausible no garantiza que α esté bien identificado.

Tres ideas para llevarse

1

Una PINN representa u_θ y deriva esa función mediante autograd.

2

La pérdida combina física, Dirichlet y Neumann con pesos explícitos.

3

Validar exige puntos nuevos, varias semillas y una referencia numérica.

$$\Delta u_\theta(\mathbf{r}) = f(\mathbf{r}) \Rightarrow r_\Omega(\mathbf{r}) = \Delta u_\theta(\mathbf{r}) - f(\mathbf{r})$$

¿Preguntas?